Android13 LED
开发指南

版本号: 1.1
发布日期: 2023.11.14

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
1.0	2023.08.17	AWA2078	添加初版。
1.1	2023.11.14	AWA2078	修改标点和文本溢出问题。

广州星翼电子科技有限公司zdyzlinux

目　录

1 概述 **1**
1.1 编写目的 1
1.2 适用范围 1
1.3 相关人员 1

2 模块介绍 **2**
2.1 相关术语介绍 2
2.2 源码结构说明 2

3 模块配置 **3**
3.1 Gpio Java 类配置 3
3.2 Gpio JNI 配置 3

4 模块接口说明 **4**
4.1 Gpio Java 接口说明 4
4.1.1 nativeWriteGpio 4
4.1.2 checkLed 4
4.1.3 setNormalLedOn 4
4.1.4 setStandbyLedOn 5
4.1.5 setNetworkLedOn 5
4.2 Gpio JNI 5
4.2.1 native_write_gpio 5

5 使用示例 **7**

6 FAQ **8**
6.1 为什么传入 setxxxxLedOn 接口的参数为 true 时，灯灭；为 false ，灯亮 8
6.2 出现 system_server: No implementation found for xxx 报错 8

广州星翼电子科技有限公司zdyzlinux

1 概述

1.1 编写目的

本文档对 android 的 LED 接口使用进行详细的阐述，让用户明确掌握 android 控制 LED 的原理，可根据自身需求进行 LED 功能定制。

1.2 适用范围

Android13

1.3 相关人员

本文档适用于所有想了解 android 平台 input LED 的相关人员。

广州星翼电子科技有限公司zdyzlinux

2 模块介绍

2.1 相关术语介绍

术语	解释说明
JNI	Java 提供的一套编程接口，用于实现 Java 程序与本地代码（如 C/C++）之间的相互调用。

2.2 源码结构说明

本模块创建一个 Gpio Java 类，以便 android 系统调用。Gpio Java 类又通过 JNI 调用 C/C++ 实现 LED 的控制。

- Gpio Java 类

路径：frameworks/base/services/core/java/com/aw/server/Gpio.java

代码结构：

```
├──── AwSystemServer.java
├──── Gpio.java   // Gpio Java 类具体实现
├──── MouseMode.java
└──── Ntc.java
```

- Gpio JNI

路径：vendor/aw/public/framework/gpio

代码结构：

```
├──── Android.mk
├──── gpio.mk      // 板级控制 gpio JNI 编译文件
└──── jni
  ├──── Android.mk  // JNI 编译文件
  └──── com_softwinner_gpio.cpp // JNI 具体实现
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3 模块配置

3.1 Gpio Java 类配置

Gpio.java 系统默认编译，无需进行配置。

3.2 Gpio JNI 配置

编译 Gpio JNI 的话，需要将 gpio.mk，加入到板级编译文件中，当前默认所有方案均会编译，编译配置在 device/softwinner/common/input/config.mk。

```
$(call inherit-product-if-exists, vendor/aw/public/framework/gpio/gpio.mk)
```

如果只想在单独方案进行编译，则需要在单独方案 input/config.mk 文件加入以下内容，并且将 common/input/config.mk 相关内容删除。

```
$(call inherit-product-if-exists, vendor/aw/public/framework/gpio/gpio.mk)
```

广州星翼电子科技有限公司zdyzlinux

4 模块接口说明

4.1 Gpio Java 接口说明

4.1.1 nativeWriteGpio

- 函数原型：private static native int nativeWriteGpio(String path, boolean value);
- 作用：给 GPIO 节点写值
- 参数：
  - path: LED 使能节点路径
  - value: true / false 对应着亮灭
- 返回：
  - 成功：返回 0。
  - 失败：返回 -1。

4.1.2 checkLed

- 函数原型：private static boolean checkLed()
- 作用：检查 LED 节点是否存在
- 返回：
  - 成功：至少存在一个 LED 节点，返回 ture。
  - 失败：一个 LED 节点均不存在，返回 false。

4.1.3 setNormalLedOn

- 函数原型：public static int setNormalLedOn(boolean on)
- 作用：使能 /sys/class/gpio_sw/normal_led/light
- 参数：
  - on: 是否使能节点

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- 返回：
  - 成功：返回 0。
  - 失败：返回 -1。

4.1.4 setStandbyLedOn

- 函数原型：public static int setStandbyLedOn(boolean on)
- 作用：使能 /sys/class/gpio_sw/standby_led/light
- 参数：
  - on: 是否使能节点
- 返回：
  - 成功：返回 0。
  - 失败：返回 -1。

4.1.5 setNetworkLedOn

- 函数原型：public static int setNetworkLedOn(boolean on)
- 作用：使能 /sys/class/gpio_sw/network_led/light
- 参数：
  - on: 是否使能节点
- 返回：
  - 成功：返回 0。
  - 失败：返回 -1。

4.2 Gpio JNI

4.2.1 native_write_gpio

- 函数原型：static jint native_write_gpio(JNIEnv *env, jobject clazz, jstring path, jboolean value)
- 作用：给节点写值
- 参数：

  - path: 节点路径
  - value: true / false 对应着亮灭

- 返回：

  - 成功，返回 0。
  - 失败，返回 -1。

广州星翼电子科技有限公司zdyzlinux

5 使用示例

控制 Normal LED 的亮灭。

```
import com.softwinner.Gpio;

...
    @Override
    public int interceptKeyBeforeQueueing(KeyEvent event, int policyFlags) {
        int keyCode = event.getKeyCode();
        final boolean down = event.getAction() == KeyEvent.ACTION_DOWN;
        if (down) {
            Gpio.setNormalLedOn(false); // 灭
        } else {
            Gpio.setNormalLedOn(true); // 亮
        }

        return super.interceptKeyBeforeQueueing(event,policyFlags);
    }
...
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6 FAQ

6.1 为什么传入 setxxxxLedOn 接口的参数为 true 时，灯灭；为 false ，灯亮

由于硬件电路的设计存在不同，部分电路给节点写 0 为灯亮，部分电路给节点写 1 为灯亮。因此出现该问题时，修改 Gpio JNI 参数即可，如下：

路径：vendor/aw/public/framework/gpio/jni/com_softwinner_gpio.cpp

- 节点写 0 为灯亮，写 1 灯灭

```
#define POWER_ON   "0"
#define POWER_OFF  "1"
```

- 节点写 1 为灯亮，写 0 灯灭

```
#define POWER_ON   "1"
#define POWER_OFF  "0"
```

6.2 出现 system_server: No implementation found for xxx 报错

在 logcat 中，可看到如下详细报错：

```
system_server: No implementation found for int com.softwinner.Gpio.nativeWriteGpio(java.lang.String, boolean)
```

说明没有将 Gpio JNI 的 gpio.mk 加入到方案中编译。

解决方法：在方案 input/config.mk 文件加入以下内容。

```
$(call inherit-product-if-exists, vendor/aw/public/framework/gpio/gpio.mk)
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

著作权声明

版权所有 ©2023 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

、、全志科技、（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。